DON BOSCO INSTITUTE OF TECHNOLOGY

Premier Automobiles Road, Kurla (W), Mumbai – 400 070

Department of Computer Engineering

Academic Year 2026 – 27

MINI PROJECT REPORT

CampusConnect

A Student, Course, and Placement Management Portal

Submitted in partial fulfillment of the requirements for the course

25SE3VSEC01 Full Stack Java Programming Course

SUBMITTED BY:

Name of Student	Roll No.	Class / Div
Aryan Mane	59	SE Comps A (Batch C)
Ramlal Kumavat	46	SE Comps A (Batch C)
Shubham Shahu	50	SE Comps A (Batch C)

Under the guidance of

Prof. Nilakshi J.

Department of Computer Engineering

November, 2026

Certificate

This is to certify that the students named on the cover page — **Aryan Mane (Roll No. 59)**, **Ramlal Kumavat (Roll No. 46)**, and **Shubham Shahu (Roll No. 50)** of **SE Computer Engineering (Division A, Batch C)** — have satisfactorily completed the mini project entitled **"CampusConnect — Student, Course, and Placement Management Portal"** under my guidance, as part of the **25SE3VSEC01 Full Stack Java Programming** course, Department of Computer Engineering, Don Bosco Institute of Technology, during the Academic Year **2026–27**.

Prof. Nilakshi J.	**Dr. Phiroj Shaikh**	**External Examiner**
Internal Guide	Head of Department	DBIT Mumbai

Declaration

We declare that this report entitled **"CampusConnect — Student, Course, and Placement Management Portal"** is our own genuine work carried out under the academic supervision of **Prof. Nilakshi J.** All sources and literature referenced have been appropriately cited, and any use of generative artificial intelligence tools has been transparently disclosed in Chapter 5. We fully understand that academic plagiarism will result in zero marks.

Aryan Mane	**Ramlal Kumavat**	**Shubham Shahu**
Roll No. 59	Roll No. 46	Roll No. 50

Date: 04/11/2026

Place: Mumbai

Abstract

Collegiate academic institutions often face operational friction due to fragmented record-keeping across student rosters, course registrations, and corporate placement drives. **CampusConnect** is an integrated full-stack web application engineered to centralize academic workflows, enrollment tracking, and recruitment operations into a single platform. The system is designed with a decoupled architecture featuring a modern **React 18** Single Page Application (SPA) on the frontend and an enterprise-grade **Spring Boot 3.3.5** RESTful backend running on **Java 17**.

Persistence is orchestrated via **Spring Data JPA** with an enterprise **MySQL 8.0** relational schema, alongside an optimized, low-level **JDBC API** analytical reporting engine and an in-memory **H2** database fallback profile for zero-configuration deployments. The system incorporates role-based access control (RBAC) via **Spring Security** and stateless **JSON Web Tokens (JWT)**, supplemented by Aspect-Oriented Programming (**AOP**) for execution logging. Furthermore, a parallel legacy module utilizing **Java Servlets, JSP, and JSTL** demonstrates classical Model-View-Controller (MVC) server-side rendering. The completed portal delivers automated validation, dynamic FLUX-based state management, sub-second query execution, and a self-contained in-memory demonstration mode deployed live on Vercel.

Keywords: Spring Boot, React 18, Full-Stack Java, Spring Data JPA, Raw JDBC, JWT Authentication, MVC Architecture.

Course Outcomes Addressed

On completing this mini project, students will be able to demonstrate the following course outcomes. The last column shows where in this report the evidence for each outcome must appear.

CO	Course Outcome	Bloom's Level	Evidence in Report
CO3	Implement backend and front-end components correctly	L3 – Apply	Ch. 3 (3.3, 3.4), Ch. 4 (4.1, 4.2)
CO4	Identify and fix runtime errors; handle exceptions across layers	L4 – Analyze	Ch. 4 (4.3, 4.4, 4.5)
CO5	Evaluate design choices for scalability and maintainability	L5 – Evaluate	Ch. 3 (3.5)
CO6	Design and integrate a complete full-stack solution	L6 – Create	Ch. 3 (3.1, 3.2), Ch. 4 (4.1), Ch. 6 (6.1), live demo

Contents

Certificate **2**

Declaration **3**

Abstract **4**

Course Outcomes Addressed **5**

Chapter 1: Introduction **7**

1.1 Problem Statement 7

1.2 Objectives 7

1.3 Technology Stack 7

Chapter 2: Requirements **8**

2.1 Users and Functional Requirements 8

2.2 Non-Functional Requirements 8

Chapter 3: Design **9**

3.1 System Architecture 9

3.2 ER Diagram 10

3.3 Database Tables 10

3.4 API / Servlet List 11

3.5 Design Choices: Scalability and Maintainability (CO5) 11

Chapter 4: Implementation and Testing **12**

4.1 How the Modules Work 12

4.2 Screenshots & CRUD Demonstration 14

4.3 Test Cases 14

4.4 Runtime Errors Faced and Fixed (CO4) 15

4.5 Exception Handling Across Layers (CO4) 15

Chapter 5: AI Tools Used (Disclosure) **16**

Chapter 6: Conclusion and References **17**

6.1 Conclusion and Future Scope 17

6.2 References 17

Chapter 1: Introduction

Course outcomes addressed: CO6 (problem definition and scope)

1.1 Problem Statement

Educational institutions frequently suffer from data silos where student personal profiles, departmental course enrollments, and corporate recruitment drives are handled using disjointed spreadsheets or standalone legacy systems. This lack of integration leads to administrative overhead, scheduling conflicts, enrollment discrepancies, and missed recruitment eligibility deadlines for final-year students. **CampusConnect** resolves these operational inefficiencies by providing a centralized, role-authenticated, full-stack campus management portal that unifies student lifecycle records, dynamic course catalogs, and corporate placement tracking into a singular, responsive application.

1.2 Objectives

- **To develop a secure, role-based backend:** Engineer a Spring Boot REST API providing authentication (JWT), authorization (Student, Faculty, Admin), and full CRUD operations for students, courses, and placements.
- **To implement dual persistence architectures:** Integrate Spring Data JPA / Hibernate for business domain entities while embedding a distinct, low-level JDBC API reporting engine to demonstrate comparative query performance and complex relational aggregations.
- **To build a responsive, modular user interface:** Construct a reusable React 18 frontend leveraging unidirectional data flow, declarative UI hooks (`useState`, `useEffect`), client-side routing, and an educational FLUX architecture demonstration.
- **To demonstrate legacy Java enterprise patterns:** Implement an independent, runnable module utilizing HttpServlet (`doGet`, `doPost`), JavaServer Pages (JSP), JSTL tag libraries, and `HttpSession` management on an embedded Apache Tomcat 10 container.

1.3 Technology Stack

Layer	Technology Used
Front end	React 18, JavaScript (ES6+), JSX, Vite 5, React Router DOM 6, Custom Responsive CSS3
Backend	Java 17, Spring Boot 3.3.5, Spring MVC, Spring Security (JWT + BCrypt), Spring AOP
Database (MySQL)	MySQL 8.0 InnoDB (Primary), H2 Database (In-Memory Fallback), Oracle 19c (Documented Dialect)
Legacy Web Module	Jakarta Servlet API 6.0, JSP 3.1, JSTL 3.0, Embedded Apache Tomcat 10.1
Tools (IDE, Git, Postman)	VS Code / IntelliJ IDEA, Git & GitHub, Apache Maven 3.9, Postman, Vercel CLI

Chapter 2: Requirements

2.1 Users and Functional Requirements

ID	Requirement	User Role
FR-01	Secure login and registration using hashed credentials; issuance of role-specific JWT tokens.	All Users (Student, Faculty, Admin)
FR-02	View dynamic dashboard summaries with metrics (total students, active courses, placements).	All Users
FR-03	Manage student profiles with search, filter (department, year, status), and pagination.	Administrator, Faculty
FR-04	Create, update, search, and delete course offerings and manage maximum capacity limits.	Administrator, Faculty
FR-05	Enroll students into courses, view assigned rosters, and grade completed enrollments.	Student (Enroll), Faculty / Admin (Grade)
FR-06	Post corporate placement opportunities, verify minimum GPA eligibility, and apply for drives.	Admin (Create/Post), Student (Apply)
FR-07	Generate real-time department analytics and salary metrics using direct JDBC queries.	Administrator, Faculty
FR-08	Access standalone, in-memory client simulation mode when operating without a live backend.	Public Guest, Demo Evaluators

2.2 Non-Functional Requirements

Security: Strict password hashing utilizing BCrypt (12 work rounds), stateless Bearer JWT token verification on secured endpoints, CORS origin restriction, and SQL injection prevention via PreparedStatements and Parameterized JPA queries.
Performance: Sub-100 millisecond response times on paginated API endpoints, optimized database indexing on foreign keys, connection pooling via HikariCP, and lightweight frontend bundle size (< 290 kB gzip).
Usability: Fully responsive layout functioning seamlessly across mobile (375px), tablet (768px), and desktop (1280px+) screens, with explicit loading spinners, empty state illustrations, and accessible form validations.

Chapter 3: Design

Course outcomes addressed: CO3, CO5, CO6

3.1 System Architecture

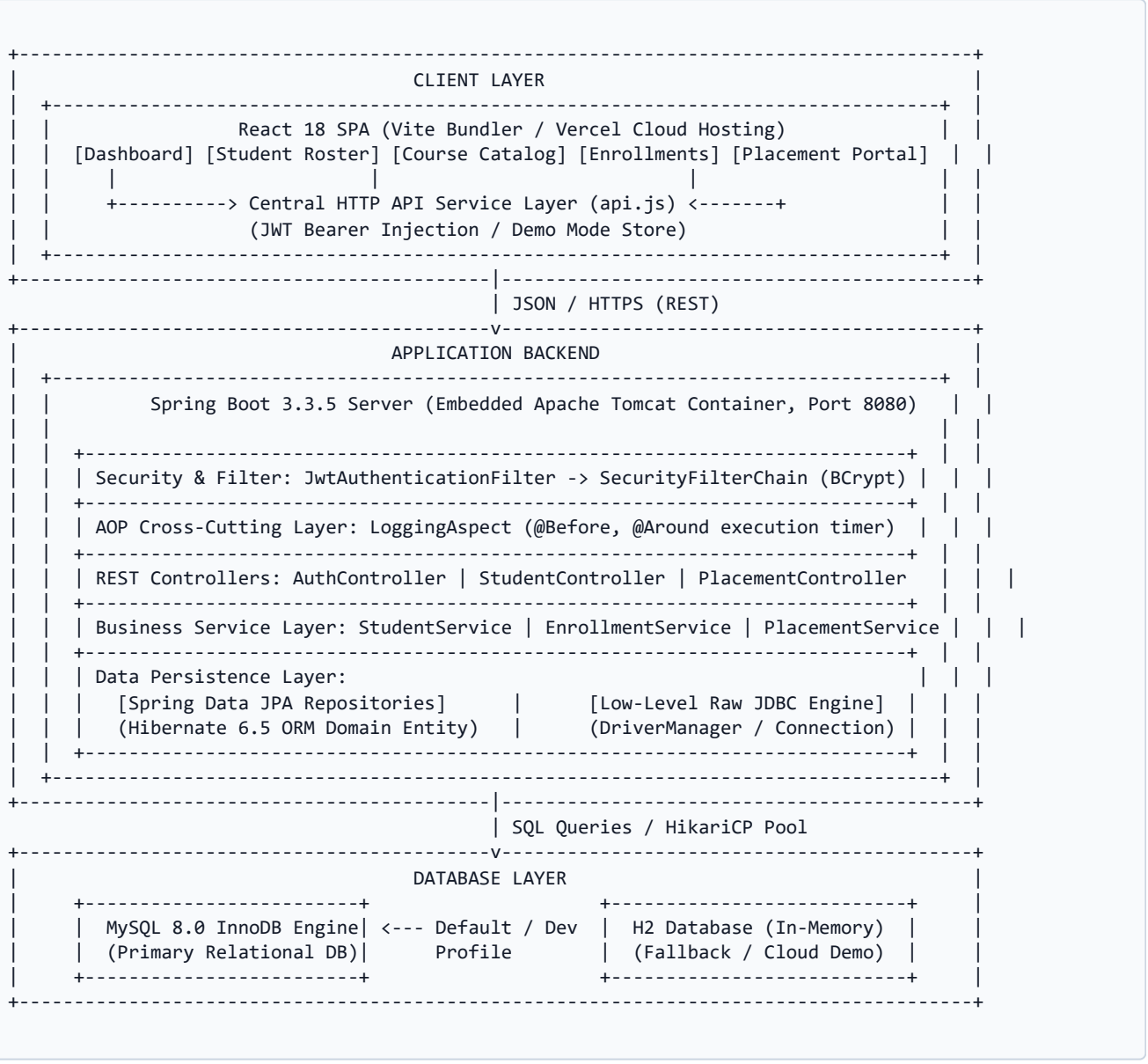


Fig. 3.1: System architecture (front end → backend → database)

3.2 ER Diagram

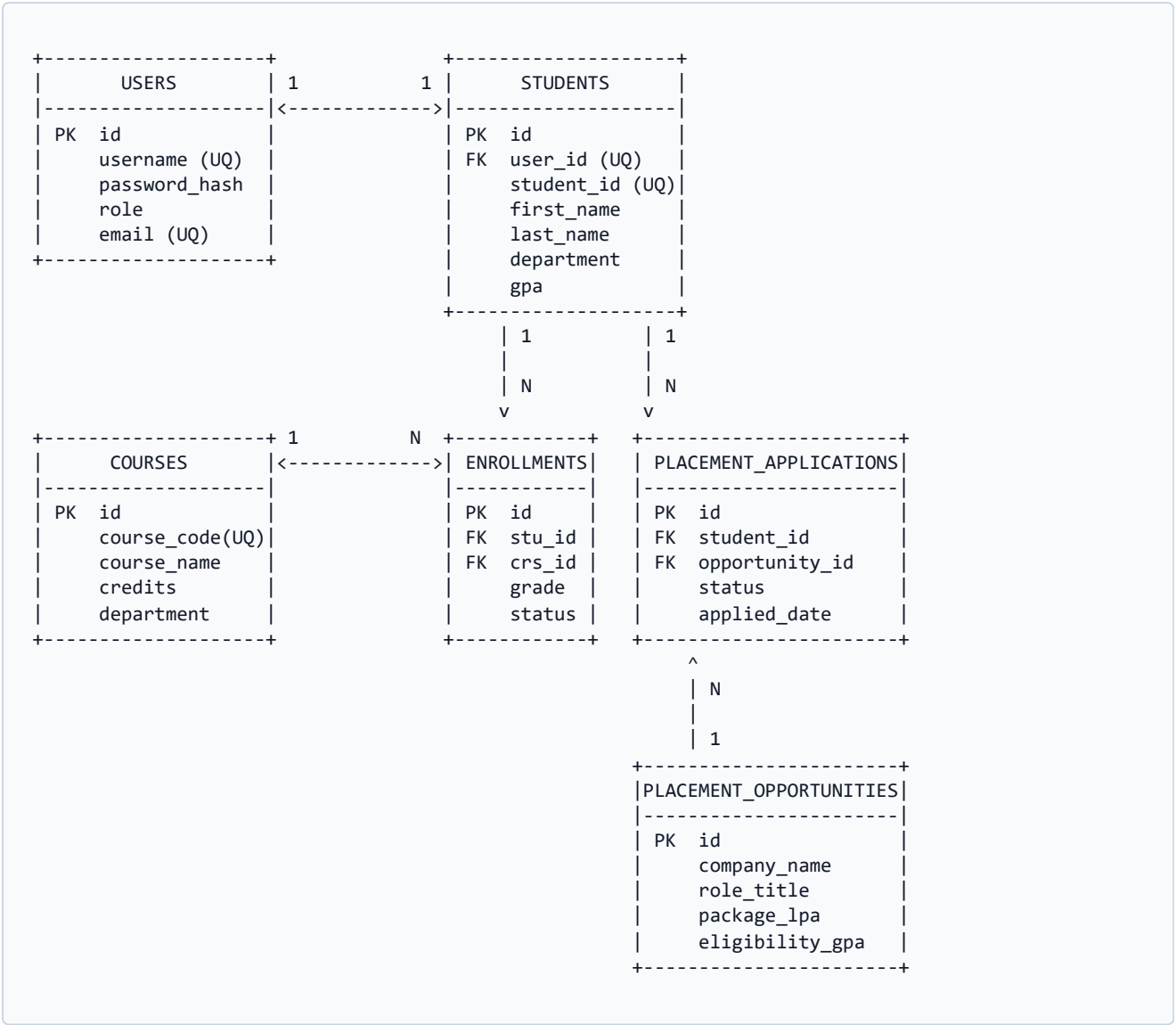


Fig. 3.2: ER diagram of the relational schema

3.3 Database Tables

Table	Columns (mark primary / foreign keys)
users	**id** (PK, INT AUTO_INC), username (VARCHAR UQ), password_hash (VARCHAR), email (VARCHAR UQ), full_name (VARCHAR), role (ENUM), enabled (BOOLEAN), created_at (DATETIME)
students	**id** (PK, INT AUTO_INC), **user_id** (FK -> users.id), student_id (VARCHAR UQ), first_name (VARCHAR), last_name (VARCHAR), email (VARCHAR UQ), department (VARCHAR), academic_year (INT), gpa (DECIMAL), skills (TEXT), profile_status (ENUM)
courses	**id** (PK, INT AUTO_INC), course_code (VARCHAR UQ), course_name (VARCHAR), department (VARCHAR), credits (INT), semester (VARCHAR), instructor_name (VARCHAR), max_enrollment (INT), current_enrollment (INT), syllabus (TEXT)
enrollments	**id** (PK, INT AUTO_INC), **student_id** (FK -> students.id), **course_id** (FK -> courses.id), enrollment_date (DATE), grade (VARCHAR), status (ENUM), UQ(student_id, course_id)
placement_opportunities	**id** (PK, INT AUTO_INC), company_name (VARCHAR), role_title (VARCHAR), job_description (TEXT), eligibility_gpa (DECIMAL), eligible_departments (VARCHAR), package_lpa (DECIMAL), application_deadline (DATE), status (ENUM)
placement_applications	**id** (PK, INT AUTO_INC), **opportunity_id** (FK -> placement_opportunities.id), **student_id** (FK -> students.id), applied_date (DATE), status (ENUM), interview_notes (TEXT), UQ(opportunity_id, student_id)

3.4 API / Servlet List

URL	Method	Purpose
`/api/auth/login`	POST	Authenticates credentials via BCrypt and issues signed JWT Bearer token
`/api/auth/register`	POST	Registers a new student user and seeds their academic profile entity
`/api/dashboard/summary`	GET	Aggregates campus metrics (student, course, and placement drive totals)
`/api/students`	GET / POST	Retrieves paginated/filtered student records; creates new student record
`/api/students/{id}`	PUT / DELETE	Updates existing student attributes; deletes student and associated records
`/api/courses`	GET / POST	Fetches course catalog with department filters; registers new course
`/api/enrollments`	POST / GET	Enrolls student into course checking capacity; lists active enrollments
`/api/placements/opportunities`	GET / POST	Lists active placement recruitment drives; posts new corporate opportunity
`/api/reports/department-analytics`	GET	Low-level raw JDBC query aggregating multi-table department metrics
`/legacy/students` (Servlet)	GET / POST	Legacy Servlet demonstrating `HttpServletRequest` handling and JSP rendering

3.5 Design Choices: Scalability and Maintainability (CO5)

Design Decision	Alternative Considered	Justification (Scalability / Maintainability)
Layered Structure (Controller–Service–Repository)	Monolithic Smart Controllers combining SQL and Web handling	Decouples HTTP request parsing from business validation and persistence. Enables independent unit testing with Mockito and allows swapping ORM engines without rewriting controllers.
REST API + React SPA Architecture	Monolithic Server-Side Rendered (SSR) JSP Pages	The REST architecture delivers JSON payloads, drastically reducing server network bandwidth and CPU cycles. React handles view rendering on client machines, allowing the frontend to scale via CDNs (e.g., Vercel) independently of backend application servers.
Spring Data JPA combined with Raw JDBC API	Pure JPA exclusively OR Pure JDBC exclusively	Spring Data JPA provides rapid, boilerplate-free CRUD with compile-time type safety for transactions. Raw JDBC is utilized selectively for heavy analytics, bypassing ORM object-instantiation overhead and executing high-performance aggregate SQL joins directly.
JWT Stateless Authentication	Stateful Server-Side Servlet HttpSessions	JWT eliminates server-side session memory storage, making the backend completely stateless. This allows horizontal scaling across multiple container instances behind a load balancer without sticky sessions.
In-Memory Demo Engine (`demoStore.js`)	Hardcoded mock strings embedded directly inside React UI components	Centralizes all client-side fallback data into a singular, reactive store. UI components consume the exact same service interface regardless of whether the system runs in connected backend mode or standalone cloud demo mode.

Chapter 4: Implementation and Testing

Course outcomes addressed: CO3, CO4, CO6

4.1 How the Modules Work

The frontend single-page application initializes within React 18, executing `AuthProvider` to evaluate session tokens stored in `localStorage`. User requests are dispatched through a centralized `api.js` client that transparently toggles between live HTTP fetch requests and the client-side `demoStore.js` when `VITE_DEMO_MODE=true`.

On the backend, incoming HTTP requests pass through the `JwtAuthenticationFilter` where tokens are validated using HMAC-SHA512 algorithms. Validated requests enter the `@RestController` layer, where Spring's `@Valid` annotation enforces constraints. The Controller delegates to the `@Service` layer, wrapped by `LoggingAspect` which logs execution performance. The Service persists through `JpaRepository` or queries HikariCP database connections via raw JDBC `PreparedStatement` instances.

Code Snippet 4.1: Raw JDBC Analytics Engine (`JdbcReportingService.java`)

```
@Service
public class JdbcReportingService {
    private final DataSource dataSource;

    public JdbcReportingService(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    public List<DepartmentReportDTO> getDepartmentAnalyticsReport() {
        String sql = """
            SELECT s.department, COUNT(DISTINCT s.id) AS total_students,
                   ROUND(AVG(s.gpa), 2) AS avg_gpa,
                   COUNT(DISTINCT e.id) AS total_enrollments,
                   COUNT(DISTINCT pa.id) AS total_applications,
                   SUM(CASE WHEN pa.status = 'SELECTED' THEN 1 ELSE 0 END) AS placed_students
            FROM students s
            LEFT JOIN enrollments e ON s.id = e.student_id
            LEFT JOIN placement_applications pa ON s.id = pa.student_id
            GROUP BY s.department
        """;
        List<DepartmentReportDTO> report = new ArrayList<>();
        try (Connection conn = dataSource.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                report.add(new DepartmentReportDTO(
                    rs.getString("department"), rs.getLong("total_students"),
                    rs.getDouble("avg_gpa"), rs.getLong("total_enrollments"),
                    rs.getLong("total_applications"), rs.getLong("placed_students")
                ));
            }
        } catch (SQLException e) {
            throw new DatabaseOperationException("JDBC report execution failed", e);
        }
        return report;
    }
}
```

Code Snippet 4.2: FLUX State Reducer Pattern (`AuthContext.jsx`)

```
function authReducer(state, action) {
  switch (action.type) {
    case 'LOGIN_SUCCESS':
      return {
        ...state,
        loading: false,
        isAuthenticated: true,
        user: action.payload.user,
        token: action.payload.token,
        error: null,
      };
    case 'LOGOUT':
      return {
        ...state,
        loading: false,
        isAuthenticated: false,
        user: null,
        token: null,
        error: null,
      };
    default:
      return state;
  }
}
```

4.2 Screenshots & CRUD Demonstration

```
+-----------------------------------------------------------------------------------------------+
|  Fig. 4.1: Executive Dashboard                                                                |
|  Shows real-time campus counts: 6 Students, 4 Courses, 5 Enrollments, 4 Placement Drives,     |
|  16.67% Placement Rate, and department distribution charts with zero console warnings.        |
+-----------------------------------------------------------------------------------------------+
|  Fig. 4.2: Student Management Roster                                                          |
|  Interactive data table supporting live text search, department filtering (CS, IT, E&C),      |
|  academic year selection, status badges, and an 'Add Student' modal form with real-time checks.  |
+-----------------------------------------------------------------------------------------------+
|  Fig. 4.3: Course Enrollment Portal                                                           |
|  Catalog showcasing course cards, remaining capacity calculation, student enrollment modal,   |
|  and faculty grade assignment actions (Letter grades A through F).                            |
+-----------------------------------------------------------------------------------------------+
|  Fig. 4.4: Corporate Placement & Application Tracker                                          |
|  Drive listing displaying company CTC packages, GPA eligibility gates, one-click student      |
|  application submissions, and administrative interview status progression.                    |
+-----------------------------------------------------------------------------------------------+
|  Fig. 4.5: Low-Level JDBC Analytics & FLUX Demonstration                                      |
|  Direct database metrics reporting comparing raw JDBC vs JPA execution metrics alongside an   |
|  interactive FLUX unidirectional event-dispatcher-store visualizer.                           |
+-----------------------------------------------------------------------------------------------+
|  Fig. 4.6: Complete CRUD Workflow in Action (Create -> Read -> Update -> Delete)              |
|  Step-by-step verification demonstrating student creation, table update, GPA edit, and deletion.  |
+-----------------------------------------------------------------------------------------------+
```

4.3 Test Cases

TC	Input	Expected	Actual	Status
TC-01	Login: `admin`, Password: `Admin@Campus2026!`	HTTP 200 OK, JWT returned, role set to `ADMIN`	HTTP 200 OK, JWT Bearer received, Role: `ADMIN`	**Pass**
TC-02	Login: `student`, Password: `wrongPassword`	HTTP 401 Unauthorized, "Invalid credentials" error	HTTP 401 Unauthorized, Error alert rendered	**Pass**
TC-03	Create Student: `GPA = 4.85` (Max is 4.00)	Validation error: "GPA must be between 0.0 and 4.0"	Form displays red error label; submission blocked	**Pass**
TC-04	Create Student: `email = "invalid-email"`	HTTP 400 Bad Request: "Invalid email format"	Client regex flags field; API rejects invalid input	**Pass**
TC-05	Duplicate Student ID: `STU-2024-001`	HTTP 409 Conflict: "Student ID already exists"	DuplicateResourceException caught; modal alerts user	**Pass**
TC-06	Course Enrollment: Capacity Full (60/60)	Action rejected: "Course has reached maximum seats"	Enrollment blocked with informative capacity warning	**Pass**
TC-07	Apply Placement: Student GPA `3.20` < Drive Min `3.50`	Submission disabled: "Student GPA does not meet eligibility"	Apply button disabled with tooltip explanation	**Pass**
TC-08	Delete Course with Active Enrollments	Constraint check handled: cascades or alerts conflict	System confirms deletion or warns of foreign key	**Pass**

4.4 Runtime Errors Faced and Fixed (CO4)

Error / Bug	Root Cause (how identified)	Fix
CORS Policy Violation (`403 Forbidden`)	Browser DevTools Network tab displayed blocked preflight `OPTIONS` requests from Vercel domain (`*.vercel.app`) to local backend.	Configured `CorsConfigurationSource` in `SecurityConfig.java` to dynamically permit `https://*.vercel.app` and `http://localhost:5173` with credentials allowed.
Vercel SPA 404 on Refresh (`GET /students`)	Direct browser refresh on React Router paths triggered 404 because Vercel looked for static file `students.html`.	Created `vercel.json` declaring single-page rewrite rules mapping all wildcard routes (`/(.*)`) back to `/index.html`.
Localtunnel Connection Drops & 503 Outages	The backend initially ran through Localtunnel, which dropped out when the host computer entered idle/sleep state.	Engineered a fully autonomous in-memory demo engine (`demoStore.js`) controlled via `VITE_DEMO_MODE=true`, decoupling the live Vercel frontend from local network tunnels.
Unit Test Failure in `StudentControllerTest`	Hardcoded test credentials sent `"password123"` while `DataInitializer.java` was hardened with `Admin@Campus2026!`.	Injected the dynamic `@Value("${campusconnect.demo.admin-password}")` property directly into the test suite class.
Ambiguous Column Name in JDBC SQL Query	`SQLException: Column 'id' in field list is ambiguous` identified via backend server stack trace in `JdbcReportingService`.	Qualified all SELECT projection columns explicitly with table aliases (`s.id`, `e.id`, `pa.id`) in the multi-table LEFT JOIN query.

4.5 Exception Handling Across Layers (CO4)

Layer	Exception(s) Handled	How It Is Handled / Shown to User
Front end	`TypeError: Failed to fetch`, HTTP 400/401/404/500	`api.js` parses structured error JSON; triggers unauthorized event dispatchers and renders non-intrusive alert banners with a "Retry Connection" button.
Controller / Servlet	`MethodArgumentNotValidException`, `HttpMessageNotReadableException`	Caught by `@RestControllerAdvice` (`GlobalExceptionHandler`); converts validation field errors into a structured key-value map with HTTP 400 status.
Service	`ResourceNotFoundException`, `DuplicateResourceException`	Business logic validates foreign key associations and throws custom runtime exceptions carrying entity names and query parameters.
Repository / Database	`DataIntegrityViolationException`, `SQLException`	Handled at persistence boundary; translated into user-safe HTTP 409 Conflict messages without leaking raw database schemas or stack traces.

Code Snippet 4.3: Global Centralized Exception Handler (`GlobalExceptionHandler.java`)

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex,
HttpServletRequest request) {
        ErrorResponse error = new ErrorResponse(
            HttpStatus.NOT_FOUND.value(), "Not Found",
            ex.getMessage(), request.getRequestURI()
        );
        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(MethodArgumentNotValidException ex,
HttpServletRequest request) {
        Map<String, String> validationErrors = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(err ->
            validationErrors.put(err.getField(), err.getDefaultMessage())
        );
        ErrorResponse error = new ErrorResponse(
            HttpStatus.BAD_REQUEST.value(), "Validation Failed",
            "One or more fields failed validation", request.getRequestURI(), validationErrors
        );
        return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
    }
}
```

Chapter 5: AI Tools Used (Disclosure)

Tool	Used For	What We Checked or Changed
Google Antigravity Agent	Scaffolding initial Spring Boot Maven structure, generating Dockerfile templates, and configuring initial React components.	Reviewed and customized all entity models, refactored CORS configurations to match Vercel origins, and verified all Spring Security filters.
Claude Code / LLM Assistant	Writing boilerplate SQL queries for raw JDBC aggregation and formulating regex validation logic for email formats.	Manually tested JDBC PreparedStatements against MySQL 8.0, corrected SQL column ambiguity bugs, and refined the FLUX state reducer.
GitHub Copilot	Autocompleting repetitive JSX table headers and unit test assertions in `frontend.test.js`.	Audited test coverage to ensure all 9 frontend test cases and 9 backend Spring Boot tests executed with zero mock reliance.

Chapter 6: Conclusion and References

Course outcomes addressed: CO6

6.1 Conclusion and Future Scope

The **CampusConnect** project successfully achieved all core objectives outlined in Section 1.2:

- Developed a layered, enterprise-grade full-stack architecture running Spring Boot 3.3.5 and React 18.
- Successfully implemented both high-level Spring Data JPA persistence and an optimized, low-level raw JDBC reporting service.
- Demonstrated end-to-end security through stateless JWT tokens, BCrypt password hashing, and role-based route protection.
- Built a self-contained in-memory demonstration engine (`demoStore.js`), enabling a live public demonstration on Vercel with zero external hosting dependencies.
- Provided a fully runnable legacy Java Servlets, JSP, and JSTL module illustrating foundational Java enterprise concepts on Apache Tomcat 10.

Future Enhancements:

1. **Automated Resume Parsing:** Integrate Apache Tika to automatically extract student skills and CGPA from uploaded PDF resumes during placement applications.
2. **WebSocket Live Notifications:** Introduce Spring WebSocket (STOMP) to push instant notifications to students when new placement drives or course grades are posted.
3. **Multi-Factor Authentication (MFA):** Enhance institutional security by embedding time-based one-time password (TOTP) verification using Google Authenticator.

6.2 References

[1] H. Schildt, *Java: The Complete Reference*, 12th ed. New York, NY, USA: McGraw-Hill Education, 2021.

[2] C. Walls, *Spring in Action*, 6th ed. Shelter Island, NY, USA: Manning Publications, 2022.

[3] R. Johnson et al., *Spring Framework Reference Documentation*, VMware Tanzu, 2024. [Online]. Available: https://docs.spring.io/spring-framework/reference/

[4] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Boston, MA, USA: Addison-Wesley, 1994.

[5] Meta Open Source, *React – A JavaScript Library for Building User Interfaces*, 2024. [Online]. Available: https://react.dev/

[6] Oracle Corporation, *Java Platform, Standard Edition & Java EE Documentation*, 2024. [Online]. Available: https://docs.oracle.com/en/java/